About GKE Agent Sandbox

Standard (/kubernetes-engine/docs/concepts/choose-cluster-mode)

Preview

This product is subject to the "Pre-GA Offerings Terms" in the General Service Terms section of the Service Specific Terms (/terms/service-terms#1). Pre-GA products are available "as is" and might have limited support. For more information, see the launch stage descriptions (https://cloud.google.com/products/#product-launch-stages).

Google Kubernetes Engine (GKE) Agent Sandbox helps you manage isolated, stateful, and single-replica workloads on GKE. It is optimized for use cases like AI agent runtimes, where untrusted, LLM-generated code must be executed in a secure and performant environment.

The GKE Agent Sandbox add-on is based on the open-source Agent Sandbox controller project (https://github.com/kubernetes-sigs/agent-sandbox/) and follows its release cycles. As a managed GKE add-on, Google manages the full lifecycle of the controller, including automatic upgrades and security patches.

This document provides a conceptual overview of GKE Agent Sandbox.

Why use GKE Agent Sandbox

GKE Agent Sandbox is built for agentic workloads that require high-level scale, extensibility, and security. Key benefits include:

- **Kernel-level isolation**: Provides strong, kernel-level isolation for untrusted, LLM-generated code using technologies like gVisor (https://gvisor.dev/).
- **Sub-second provisioning**: Offers an out-of-the-box mechanism to provide sandboxes significantly faster than standard Kubernetes Pod scheduling allows (typically <1s).
- **Cloud-native extensibility**: Leverages the power of the Kubernetes paradigm and the managed infrastructure of GKE.

By providing a declarative, standardized API, GKE Agent Sandbox offers a single-container experience that provides isolation and persistence characteristics similar to a virtual machine (VM), built entirely on Kubernetes primitives.

Common use cases for Agent Sandbox

Use GKE Agent Sandbox for workloads that require isolation, persistence, and a stable identity. Example use cases include:

- **AI agent runtimes**: Safely execute untrusted code in an environment isolated by security-focused runtimes like gVisor (https://gvisor.dev/).
- **Development environments**: Provide developers with persistent, isolated cloud-based coding environments.
- **Notebooks and research tools**: Host single-container sessions for interactive tools like Jupyter Notebooks.
- **Stateful single-Pod services**: Run applications that need a stable identity and storage without the complexity of a `StatefulSet`.
- **Programmatic environment management**: Use provided client library SDKS, such as the Agent Sandbox Python SDK (https://github.com/kubernetes-sigs/agent-sandbox/tree/main/clients/python/agentic-sandbox-client), to request and manage sandboxes directly from your application logic without managing Kubernetes YAML.

How GKE Agent Sandbox works

GKE Agent Sandbox uses a custom controller and several Kubernetes Custom Resource Definitions (CRDs) to manage the lifecycle of sandboxed environments.

Core architecture

- **Sandbox CRD** (/kubernetes-engine/docs/reference/crds/agentsandbox): The primary resource that represents a single, stateful Pod. It manages stable hostnames, network identity,

and persistent storage.

- **Sandbox Router**: A component that provides a stable endpoint and tunnels traffic to the appropriate Sandbox Pods, abstracting the underlying networking complexity.
- **Integration with Pod snapshots**: GKE Agent Sandbox integrates with the GKE Pod snapshots (/kubernetes-engine/docs/how-to/agent-sandbox-pod-snapshots) feature to allow pausing and resuming workloads by saving and restoring the full state of a container.

Claim Model

The **Claim Model** is a key feature that separates the user's request for an environment from the specific implementation details, such as where and how the workload is provisioned. Unlike a standard Kubernetes `StatefulSet`, the Claim Model lets you request a sandbox without needing to manage the underlying Pod or storage configurations directly.

The Claim Model is managed using the `SandboxClaim` (/kubernetes-engine/docs/reference/crds/agentsandbox#sandboxclaim) and `SandboxTemplate` (/kubernetes-engine/docs/reference/crds/agentsandbox#sandboxtemplate) CRDs, and works as follows:

1. Users or applications request a Sandbox by creating a `SandboxClaim` that references a `SandboxTemplate`.
2. The controller handles the mapping of the claim to an actual Sandbox instance, offering flexible backend management. This allows the system to reuse existing Sandboxes or allocate from a pool.

Warm Pools

The **Warm Pool** feature is designed to minimize startup latency, which is critical for interactive AI agent scenarios. This feature allows the Agent Sandbox to provide execution environments in less than one second, significantly faster than typical Pod scheduling. The feature is managed using the `SandboxWarmPool` (/kubernetes-engine/docs/reference/crds/agentsandbox#sandboxwarmpool) CRD and works in the following way:

1. A `SandboxWarmPool` maintains a set of pre-warmed Pod instances in a ready state.

2. When a `SandboxClaim` is made, the controller instantly assigns a Pod from the pool instead of waiting for a new Pod to pull images and start from scratch.
3. When combined with Pod snapshots, warm pools provide fast, "instant-on" capabilities by restoring Pods from a pre-configured state.

Network isolation

GKE Agent Sandbox implements a **Default Deny** network security posture for all sandboxed environments. This ensures that untrusted code executed inside a sandbox cannot access unauthorized internal networks or the GKE control plane by default. You can define specific network restrictions and allowed egress or ingress rules within your `SandboxTemplate` to provide fine-grained security for agentic workloads.

Programmatic access with SDKs

AI engineers can consume GKE Agent Sandbox resources programmatically using provided client libraries. For example, the Python SDK (https://github.com/kubernetes-sigs/agent-sandbox/tree/main/clients/python/agentic-sandbox-client) provides a high-level interface that abstracts the underlying `SandboxClaim` and `SandboxTemplate` configurations. This lets you create and interact with isolated environments directly from your Python-based agent frameworks like LangChain or the Vertex AI Agentic SDK.

Limitations and requirements

GKE Agent Sandbox has the following limitations and requirements:

- **Cluster version**: Requires GKE version 1.30.2-gke.1394000 or later for full feature support (including snapshots).
- **Infrastructure requirements**: Optimized for specific node configurations (such as N2 machine types) and requires the Agent Sandbox controller to be installed and configured on the cluster.
- **Isolation runtimes**: While it supports multiple runtimes, it is primarily intended to be used with security-hardened runtimes like gVisor.

- **Underlying features availability**: Some underlying features, such as GKE Pod snapshots (/kubernetes-engine/docs/concepts/pod-snapshots), might be in Preview or have specific regional availability.

What's next

- Learn more about isolating AI code execution with Agent Sandbox (/kubernetes-engine/docs/how-to/sandbox-pods).
- To learn how to use Pod snapshots with Agent Sandbox, see Save and restore Agent Sandbox environments with Pod snapshots (/kubernetes-engine/docs/how-to/agent-sandbox-pod-snapshots).
- For the underlying open-source implementation, see the Agent Sandbox GitHub project (https://github.com/kubernetes-sigs/agent-sandbox/).
- For example runtimes and YAML configurations for scenarios such as code execution or computer use, see the Agent Sandbox examples (https://github.com/kubernetes-sigs/agent-sandbox/tree/main/examples).
- To interact with sandboxes programmatically, see the Agent Sandbox Python SDK README (https://github.com/kubernetes-sigs/agent-sandbox/blob/main/clients/python/agentic-sandbox-client/README.md) on GitHub.

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-04-06 UTC.